Springarens vandring på schackbrädet

Beskrivning

- Programmet ska likna spelet “Knight’s Tour”
- Användaren ska kunna välja mellan att se datorn spela spelet, göra egna drag, eller se statistik på 1000 drag som datorn har tagit.
- Programmet kommer att ta user inputs i terminalen skriva ut brädet i ett separat fönster

Algoritm

1. Programmet skriver ut ett tomt bräde
2. Frågar om startposition
3. Upprepa följande tills användaren vill sluta:
    a. Skriv ut det uppdaterade brädet
    b. Skriv ut meny
    c. Fråga om menyval
    d. exekvera menyvalet
        i. **1 - Play the game**: Spelaren väljer var den vill flytta hästen, tills hästen inte kan flytta längre, då tas spelaren tillbaka till menyn
        ii. **2 - Watch computer play:** Valen väljs slumpmässigt och spelaren klickar enter för att se nästa drag, tills hästen inte kan flyttas längre, då tas spelaren tillbaka till menyn
        iii. **3 - See statistics from 1000 moves:** Spelaren får en lista på rundor spelade av datorn, där den spelar om och om igen tills den har gjort 1000 drag, listan är sorterad efter högsta poäng med en kolumn för startposition för att ta reda på de bästa startpositionerna.
        iv. **4 - Exit**: Spelaren avslutar spelet

Datastrukturer

Varje gång man väljer en startposition så skapas ett nytt objekt som representerar den rundan, där namnet på objektet är startpositionen (t.e.x A8 eller E4)

Det finns en dictionary med alla instanser, alltså alla rundor som har spelats, där Key är objektets namn(start-koordinater), och Value är objektet.

Klasser och metoder

class Round:
"""
Attributes:

allowed_positions: list of coordinates, where the knight is allowed to move, the first one is up to the right from the knight. The symbols will be a-h.
previous_positions: A sorted list of coordinates, where the first coordinate is the first move, the second is the second move… The symbols on the board will be 1, 2, 3, 4…
board_matrix: A matrix containing every line, letter and number needed to print the board. The board will be a 12x12 grid with Letters A-H at the bottom and numbers 1-8 on the left side, similar to a chess board. (example - image 1 at the bottom page)

current_position: The square the knight is standing on (coordinate)

"""
def __init__(self, current_position):
"""
Creates a new Round, with current_postition as starting position.

allowed_positions: A list of coordinates, where the knight is allowed to move, the first one is up to the right from the knight. The symbols will be a-h.
previous_positions: A sorted list of coordinates, where the first coordinate is the first move, the second is the second move… The symbols on the board will be 1, 2, 3, 4…
board_matrix: A matrix containing every line, letter and number needed to print the board. The board will be a 12x12 grid with Letters A-H at the bottom and numbers 1-8 on the left side, similar to a chess board. (example - image 1 at the bottom page)

:param current_position: The square the knight is standing on (coordinate)
"""

def __str__(self):
"""
Prints the board, will be used as a graphical update function, meaning this will use all of the position attributes.
:return: string with the entire board
"""

def get_move(self):
"""
prints a message asking for a move (number for every allowed move, at most a-h)

gets input from user.
:return: The coordinate of the move choice. (Example, the player
"""

def calculate_allowed_moves(self):
“”“
calculates what moves the user can make, updates the allowed move list
:return: nothing
“”“

def make_move(self, new_position):
"""
¨'*
new_position.
:param new_position: The position the player wants to move to.
:return: nothing
"""

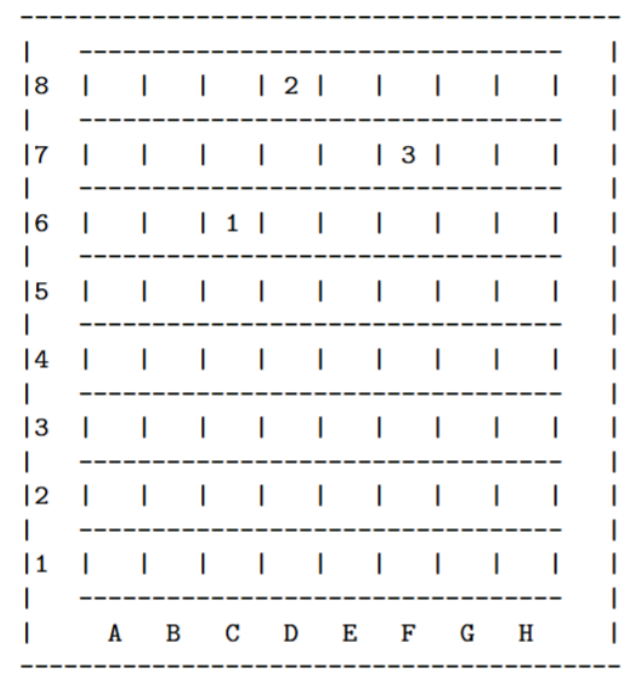
Funktioner:

def update_statistics():
“”“
Uses the dictionary of all rounds to make a list of the rounds the computer plays in the “make 1000 random moves” choice.
:return: nothing
“”“

def print_statistics():
“”“
Prints the 1000 games played, with the most amount of moves at top.
:return: nothing
“”“
def get_int_input(prompt_string):
"""
Used to get an int from the user, asks again if input is not convertible to int
:param prompt_string: the string explaining what to input
:return: the int that was asked for
"""
def menu():
"""
Used to display the menu:
What would you like to do?
1 - Play the game
2 - Watch computer play
3 - See statistics from 1000 moves
4 - Exit
:return: (nothing)
"""

```
def execute(choice):
 """
 Used to execute the option that the user chose
 :param choice: an int corresponding the the chosen option
 :return: (nothing)
 """
```

Billagor



Billaga 1 - board-matrix, exempel vid användning av att printa ett objekt med __str__ funktion.